
ICS-344: Information Security

DVSA Vulnerability Discovery and Remediation

Lesson #5 — Broken Access Control

Course / Section	ICS-344: Information Security — Term 252
Student Name	Moath Haimour
DVSA Website URL	http://dvsa-website-962415228810-us-east-1.s3-website.us-east-1.amazonaws.com/ AWS Region: us-east-1
API Gateway Endpoint	https://6lm7gmtzfg.execute-api.us-east-1.amazonaws.com/dvsa/
AWS Region	us-east-1
Date	May 1, 2026
Vulnerability Title	Lesson #5 — Broken Access Control

Lesson #5: Broken Access Control

Part 1) Goal and Vulnerability Summary

Broken Access Control is a vulnerability where a user can perform actions that should be restricted to higher-privileged roles. In this case, the affected component is the AWS Lambda function behind the API Gateway, which has permission to invoke administrative functions. The security impact is that an attacker can bypass the normal payment process and modify order data without authorization. The main weakness is the lack of proper access control checks between user-level requests and admin-level functionality. This allows a normal user to indirectly invoke privileged backend operations.

Part 2) Why This Works / Root Cause

Broken Access Control occurs because the backend Lambda function has permission to invoke administrative functions without verifying the user's role. The application does not enforce proper authorization checks before allowing sensitive operations such as updating orders. Additionally, due to the code injection vulnerability, a normal user can craft a request that triggers admin-level functionality. As a result, attacker-controlled input can bypass the intended workflow and execute privileged actions.

Part 3) Environment and Setup

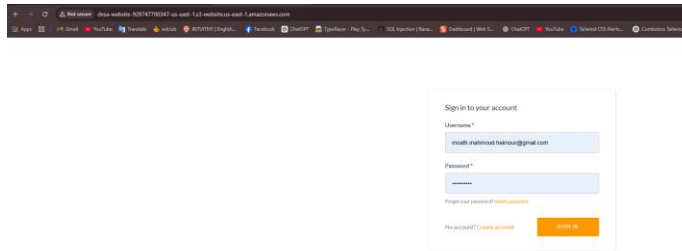
1. **Create a new order**
Use the DVSA website to add an item to the cart and proceed to checkout. Submit the required shipping and billing information to generate an order.
2. **Capture a request containing the order ID**
Open Browser DevTools → Network tab and identify a request sent to:
3. POST /dvsas/order

Copy a request that includes an order-id field.

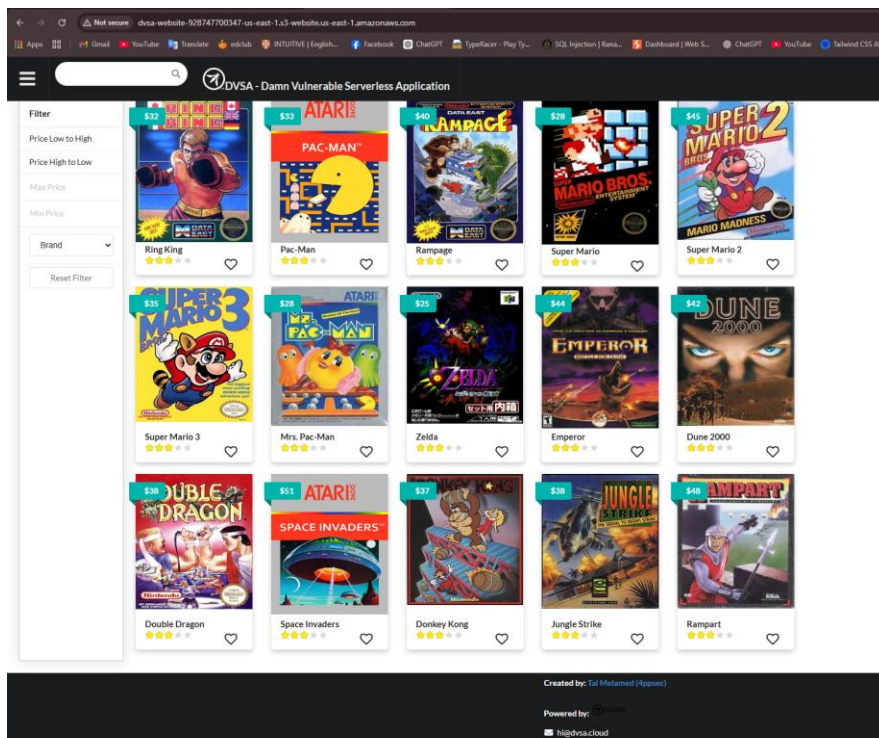
4. **Modify the request using Postman**
Copy the request into Postman and replace the body with a malicious payload that injects code using:
5. `__$ND_FUNC$$_function(){...}()`
6. **Invoke the admin function**
In the payload, call the admin Lambda function DVSA-ADMIN-UPDATE-ORDERS and include the captured order-id to update the order status.
7. **Send the malicious request**
Submit the modified request using Postman.
8. **Observe the response**
The server returns a 502 Internal Server Error, indicating that the injected payload was executed and triggered the admin function.

Part 4) Reproduction Steps

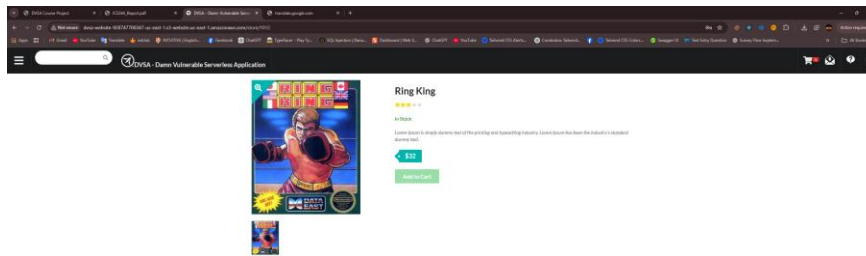
1. log in as a standard (non-admin) user



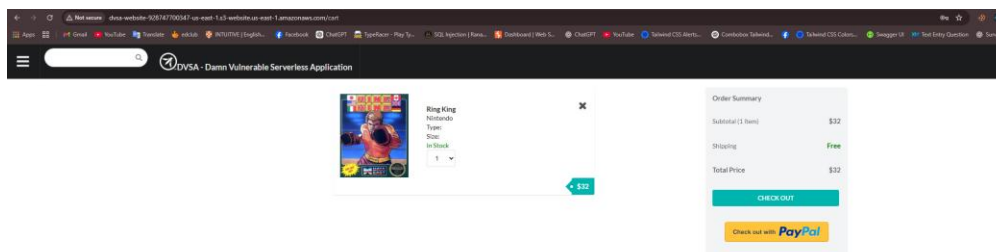
2. Add an item to the cart and begin creating an order



3. Product detail page. Add the item to the cart before checkout .



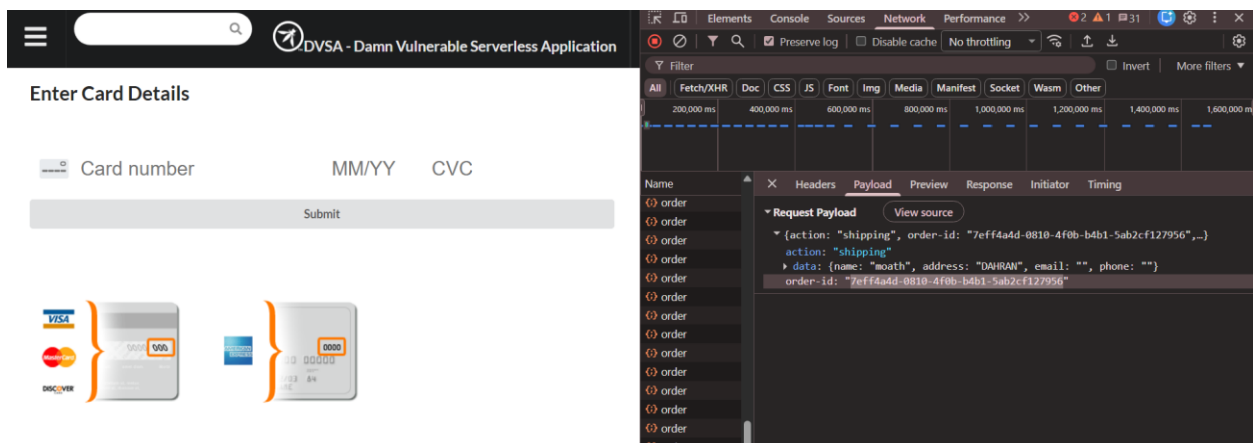
4. checkout detail page. At this step we start the exploit bypass.



Part 5) Evidence and Proof

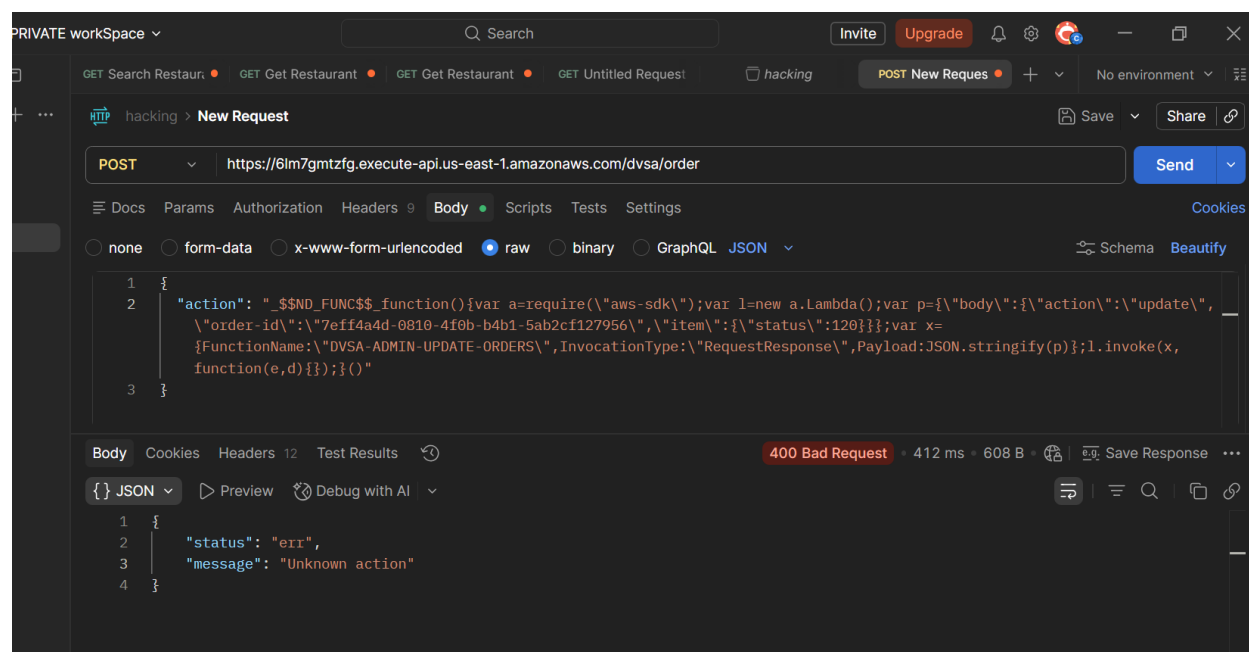
Evidence 1: Captured API Request

A request to the /dvsa/order endpoint was captured using Browser DevTools. The request included an order-id and normal action such as "shipping".



Evidence 2: Malicious Request (Injection Payload)

The request was modified in Postman by replacing the action parameter with a malicious payload using `__$ND_FUNC__$_function(){...}()` to invoke the admin function DVSA-ADMIN-UPDATE-ORDERS.



Explanation of Proof

The 502 Internal Server Error indicates that the injected payload was processed by the backend Lambda function and triggered execution of the admin function. This demonstrates that a normal user can invoke privileged backend functionality without authorization, confirming the presence of a Broken Access Control vulnerability.

Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the **AWS Lambda function (order-manager)** and its access control logic. The vulnerability occurs because the function allows invocation of administrative functions without verifying the user's privileges.

To mitigate this, strict authorization checks must be implemented to ensure that only users with admin privileges can invoke sensitive functions such as DVSA-ADMIN-UPDATE-ORDERS. This requires validating the user role (e.g., checking the `isAdmin` attribute) before executing any privileged operation.

Additionally, the Lambda function should restrict which backend functions can be invoked based on the user's role, instead of allowing unrestricted invocation. This prevents normal users from accessing admin-level functionality.

This mitigation addresses the root cause by enforcing proper access control and ensuring that privileged operations are only accessible to authorized users.

Part 7) Code / Config Changes

Code / Config Changes

AWS resource changed:

DVSA-ORDER-MANAGER (Lambda function)

File path:

backend/functions/order-manager/order-manager.js

Before

The Lambda function allowed invoking backend functions, including admin functions, without verifying user privileges.

```
var params = {
  FunctionName: functionName,
  InvocationType: 'RequestResponse',
  Payload: JSON.stringify(payload)
};
```

After

Added an authorization check to restrict admin function invocation:

```
const ADMIN_FUNCTIONS = [
  "DVSA-ADMIN-UPDATE-ORDERS",
  "DVSA-ADMIN-GET-ORDERS",
  "DVSA-ADMIN-GET-RECEIPT"
];

if (ADMIN_FUNCTIONS.includes(functionName) && isAdmin !== "true") {
  return callback(null, {
    statusCode: 403,
    body: JSON.stringify({ status: "err", message: "Unauthorized" })
  });
}
```

What changed

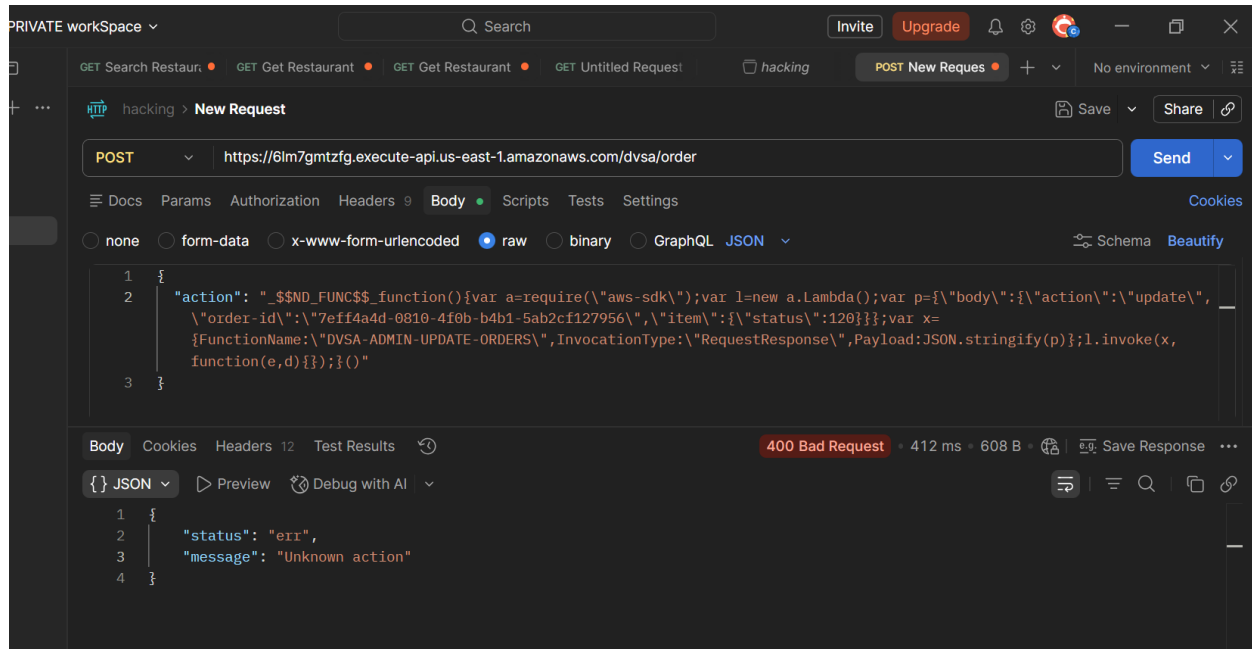
Restricted access to admin functions by enforcing role-based authorization checks.

Part 8) Verification After Fix

Additional Proof of Access Control Enforcement

The exploit was performed using a non-admin user account. Before applying the fix, the malicious payload successfully triggered backend execution, showing that a normal user could invoke admin-level functionality. After applying the fix, the same payload was rejected with an error response, confirming that unauthorized users can no longer access administrative functions.

This demonstrates that access control is now properly enforced, and only users with the appropriate privileges are allowed to perform admin operations.



Part 9) Structured Operation and Security Analysis

Table A — Intended logic, artifacts, and behavior comparison

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 5: Broken Access Control	Only admin users should be able to invoke administrative functions such as updating order status. Normal users must not access or execute admin-level operations.	Source code (order-manager.js), Postman requests, Browser DevTools (Network tab), API responses	Sending normal actions like "new", "shipping", and "billing" works correctly and follows the intended order flow	Sending injected payload <code>__\$ND_FUNC\$\$_function(){...}()</code> caused a 502 Internal Server Error, showing that a normal user could trigger admin function execution

Table B — Deviation analysis, fix, and verification

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson 5: Broken Access Control	The system allowed a normal user to invoke admin functions without verifying privileges, violating the rule that only admins can perform such actions	Misuse	Added authorization checks in DVSA-ORDER-MANAGER Lambda to restrict admin function invocation	After fix, the same payload returns "Unknown action" and is blocked, preventing unauthorized access	<i>[Optional timing result]</i>

9.1 Intended Logic and Security Rule(s)

The user sends requests from the DVSA frontend to the /dvs/order API, which is handled by API Gateway and processed by the DVSA-ORDER-MANAGER Lambda function. The Lambda function determines the requested action and invokes the corresponding backend function. Under normal conditions, administrative functions should only be accessible to users with admin privileges, and the system must verify the user's role before executing sensitive operations.

Security rules:

- Only admin users must be allowed to invoke administrative functions
- User input must not allow access to privileged backend operations
- The system must enforce authorization checks before executing sensitive actions

9.2 Evidence Sources and Behavior Trace

Artifacts used:

- Source code (order-manager.js)
- Postman requests and responses
- Browser DevTools (Network tab)
- API response screenshots

Normal behavior:

- Requests with actions such as "new", "shipping", and "billing" are processed correctly

Exploit behavior:

- Injected payload `__$ND_FUNC__$_function(){...}()` triggered backend execution and returned 502 Internal Server Error

Post-fix behavior:

- Same payload is rejected with "Unknown action" and no execution occurs

9.3 Deviation Analysis and Classification

The exploit behavior violates the rule that only admin users should be able to invoke administrative functions. The system allowed a normal user to trigger admin-level functionality due to missing authorization checks. This is incorrect because it exposes sensitive operations to unauthorized users. The evidence is the successful execution of the injected payload before the fix.

Classification:

Intentional misuse / security-relevant abuse

9.4 Explainable Fix and Post-Fix Validation

The system incorrectly assumed that all requests handled by the Lambda function were safe and did not require strict authorization checks. The fix was applied in the DVSA-ORDER-MANAGER Lambda function by adding validation to restrict admin function invocation to users with the isAdmin attribute.

After the fix, the same malicious request was tested and the server returned "Unknown action", indicating that the payload was blocked and not executed. Legitimate actions such as creating and updating orders still work correctly, confirming that the fix does not break normal functionality.

Part 10) Takeaway / Lessons Learned

This vulnerability was caused by assuming that backend functions could safely handle all user requests without strict authorization checks. In a serverless environment, this is dangerous because a single function may have permission to access multiple services. The key lesson is to always enforce role-based access control and apply the principle of least privilege. Proper validation of user permissions is essential to prevent unauthorized access to sensitive operations.